

**420-414-H26
Infonuagique**

**Épreuve finale
Conception et déploiement infonuagique**

Prénom & Nom :

Matricule :

Signature :

Contexte et mandat

*Vous êtes mandaté en tant que **développeur·euse / ingénieur·e cloud junior** pour accompagner l'entreprise **VanLife**, qui propose une plateforme de location de vans aménagés. Cette application permet aux utilisateurs de consulter des offres de location, et aux administrateurs d'accéder à un tableau de bord présentant différentes statistiques (ventes, revenus, etc.).*

Cette application est composée de :

- *Un frontend (React / React Router)*
- *Une API (NestJS)*
- *Une base de donnée (MySQL)*

Actuellement, l'application existe sous forme de code source (frontend et API), mais n'est pas encore adaptée à un environnement de production infonuagique. Votre mandat consiste à :

- *Conteneuriser les différentes composantes de l'application*
- *Publier les images sur Docker Hub*
- *Déployer l'application à l'aide d'un orchestrateur (Kubernetes ou Amazon ECS)*
- *Déployer la base de données dans un environnement externe*
- *Concevoir une architecture hautement disponible et résiliente*
- *Documenter vos choix dans un rapport structuré*

Description du projet et exigences techniques

VanLife vous fournit deux dépôts GitHub contenant le code source :

- [dépôt du frontend](#)
- [dépôt de l'API](#)

*Vous devrez déployer l'application **VanLife** en respectant les exigences suivantes :*

Conteneurisation

- Forker les deux dépôts fournis (frontend et API)
- Créer les fichiers nécessaires à la conteneurisation :
 - Dockerfile
 - .dockerignore
 - Modifier le fichier de configuration nginx.conf pour le frontend (dépendamment de l'environnement de déploiement choisi)
- Construire les images Docker
- Publier les images sur *Docker Hub*

Déploiement

Vous devrez déployer votre application en utilisant **une des deux approches suivantes** :

Option A – Kubernetes

- Déployer sur le cluster **Kubernetes** du département
- Créer les manifestes nécessaires (Deployments, Services, etc.)
- Utiliser une machine virtuelle fournie pour héberger la base de données.

Option B – Amazon ECS

- Déployer sur **Amazon ECS**
- Configurer les ressources nécessaires (cluster, services, tâches)
- Utiliser **Amazon RDS** pour la base de données

Base de données

La base de données doit être déployée **en dehors de votre environnement applicatif** :

- ECS → Amazon RDS
- Kubernetes → votre propre VM fournie dans les serveurs du département

Gestion des ressources

Vous devrez créer et organiser les ressources nécessaires pour :

- Assurer le bon fonctionnement de l'application
- Permettre l'accès aux différentes composantes (frontend, API, base de données)
- Garantir une certaine **tolérance aux pannes**

Modalités de l'épreuve

L'épreuve est composée de deux parties :

- **Partie I - Rapport d'implémentation**
- **Partie II - Démonstration de l'implémentation**

Partie I – Rapport d'implémentation

Pour ce volet, vous devrez produire un rapport détaillé présentant votre architecture et vos choix techniques. Ce rapport doit inclure les sections suivantes :

1. Introduction
2. Références des dépôts et artefacts
3. Architecture globale
4. Conteneurisation
5. Infrastructure réseau
6. Ressource de calcul et orchestration
7. Sécurité
8. Haute disponibilité et résilience
9. Déploiement
10. Limites et améliorations
11. Bonus (optionnel)
12. Annexes (optionnel)

Un gabarit du rapport contenant la structure globale attendue ainsi qu'une courte description des éléments à développer dans chaque section vous est fourni (*gabarit-rapport.docx*).

Partie II – Démonstration de l'implémentation

Pour ce volet de l'épreuve, vous devrez présenter votre implémentation lors d'une démonstration d'environ 20 minutes.

La démonstration pourra se faire :

- En classe, lors de la dernière séance, ou
- Sur rendez-vous individuel

L'objectif de cette démonstration est de **valider la cohérence entre le rapport et l'implémentation réelle** ainsi que votre **compréhension des outils et concepts utilisés**.

Éléments à démontrer

Durant la démonstration, vous devez être en mesure de présenter les éléments suivants :

1. Architecture globale
Expliquer vos choix (Kubernetes ou ECS, organisation des services, stratégie de déploiement)
2. Étapes d'implémentation
Expliquer les étapes pour arriver au produit final présenté.
3. Conteneurisation
Présenter vos images Docker et expliquer vos Dockerfiles
4. Orchestration
Montrer les ressources déployées et expliquer leur rôle :
 - Kubernetes : pods, deployments, replicaset, services, etc...
 - ECS : cluster, services, définition de tâche, tâches en cours d'exécution

5. Explication technique

Décrire brièvement l'infrastructure réseau, les stratégies mises en place pour assurer la sécurité, la haute disponibilité et la résilience de l'application tout en faisant le lien avec ce qui a été décrit dans votre rapport de conception.

6. Démonstration fonctionnelle

Montrer que votre application est accessible et fonctionnelle (accès au frontend, communication entre le frontend, l'API et la base de données).

7. Dépôts et configuration

Présenter les dépôts Git, les fichiers de déploiement et expliquer l'organisation du projet.

Dates importantes et modalités de remise

- Le rapport doit être déposé sur Léa **au plus tard le 15 mai (fin de journée)**.
- La date limite de prise de **rendez-vous pour la démonstration est fixée au 15 mai**.
- **Toutes les démonstrations devront être complétées au plus tard le 22 mai**.

Il est de votre responsabilité de vous inscrire ou de planifier votre rendez-vous dans les délais prescrits. **Aucune démonstration ne sera acceptée après la date limite.**

Grille de correction

Critère	Points
Partie I - Rapport de conception	100
1. Références des dépôts et artefacts <i>Tous les liens présents, fonctionnels, bien organisés</i>	5
2. Architecture globale <i>Architecture claire, cohérente, schéma pertinent, flux bien expliqués</i>	15
3. Conteneurisation <i>Explications détaillées, choix techniques justifiés</i>	10
4. Infrastructure réseau <i>Description complète, concepts bien maîtrisés</i>	10
5. Ressources de calcul et orchestration <i>Bonne compréhension (ECS ou K8s), explications structurées</i>	15
6. Base de données <i>Intégration claire, connexion bien expliquée</i>	5
7. Sécurité <i>Bonnes pratiques pertinentes, justification claire</i>	10
8. Haute disponibilité et résilience <i>Stratégies pertinentes, cohérentes et bien expliquées</i>	10
9. Qualité globale du rapport <i>Rapport clair, structuré, rigoureux</i>	20
TOTAL	100

Critère	Points
Partie II - Démonstration	100
1. Fonctionnalité du déploiement <i>Application fonctionnelle</i> <i>Communication frontend ↔ backend ↔ BD fonctionnelle</i>	20
2. Maîtrise de la conteneurisation <i>Bonne compréhension de la méthode de conteneurisation</i>	10
3. Maîtrise de l'orchestration <i>Excellente maîtrise (K8s ou ECS), démonstration claire</i>	20
4. Explication de l'architecture <i>Explication claire et structurée</i>	20
5. Cohérence rapport / implémentation <i>Bonne cohérence entre le rapport et l'implémentation</i>	15
6. Qualité de la présentation <i>Présentation fluide, professionnelle, bien structurée</i>	15
Bonus (jusqu'à + 10 points)	
<i>Terraform fonctionnel + expliqué : +5</i> <i>Application personnelle complète et pertinente : +5</i>	
TOTAL	100